Queue

Queue

- Queue is a linear data structure that permits insertion of an element at one end and deletion of an element at the other end.
- The end at which the deletion of an element take place is called front, and the end at which insertion of a new element can take place is called rear.
- The deletion or insertion of elements can take place only at the

front and rear end of the list respectively

- first element that gets added into the queue is the first one to get removed from the list. Hence, Queue is also referred to as First-In-First-Out (FIFO) list.

Queue Data Structure

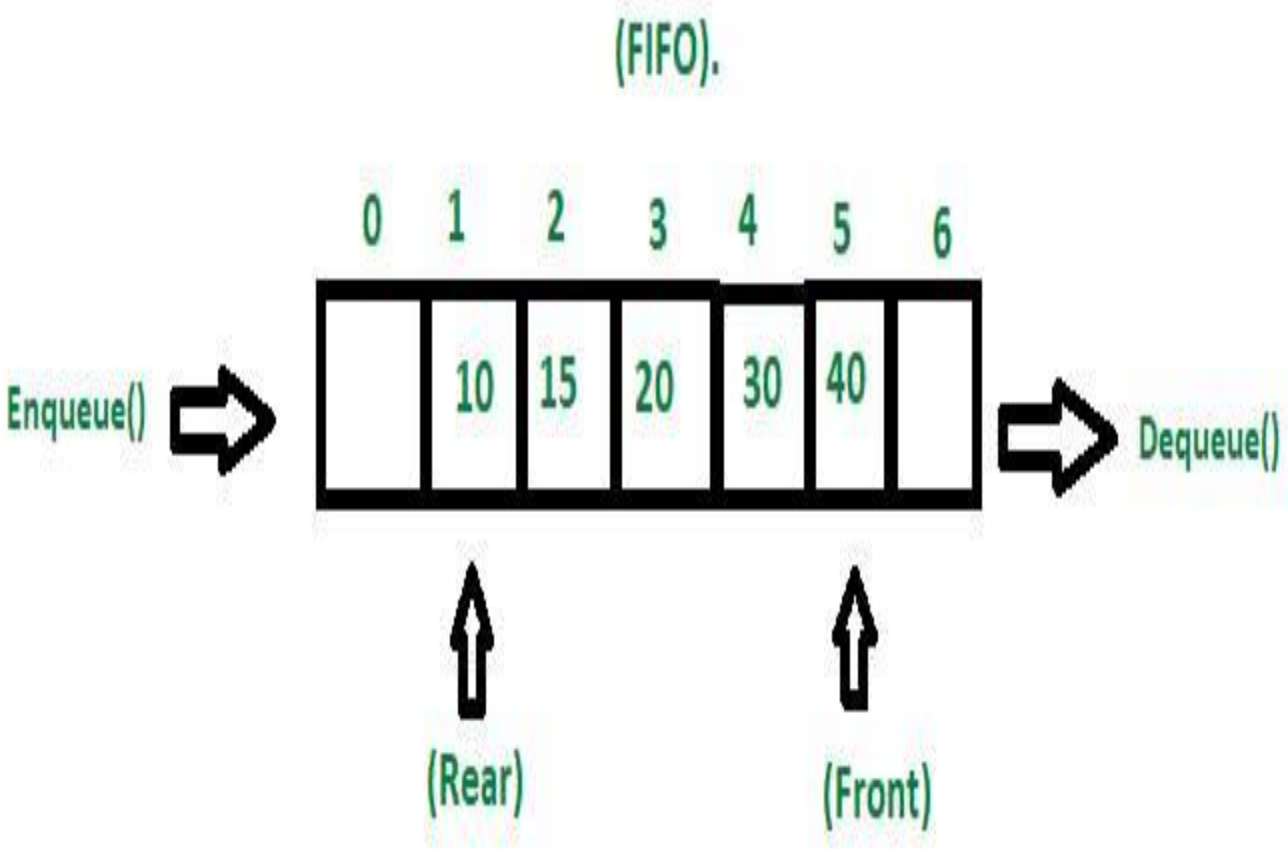
Queue

- Consider a railway reservation booth, at which we have to get into the reservation queue. New customers got into the queue from the rear end, whereas the customers who get their seats reserved leave the queue from the front end.
- It means the customers are serviced in the order in which they arrive the service center (i.e. first come first serve type of service)

Operations on Queue

- Enqueue – add/ store an item to the queue.insertion will be done at rear
-
- dequeue – remove an item from the queue.deletion will be done at front

Queue



Queue

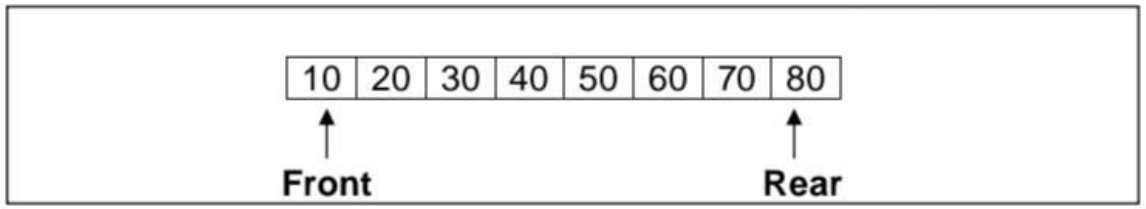


Fig. (1) : Pictorial representation of a Queue

In fig (1), **10** is the first element and **80** is the last element added to the Queue. Similarly, **10** would be the first element to get removed and **80** would be the last element to get removed.

Queue

Figures 2(a) to 2(d) shows queue graphically during insertion operation :

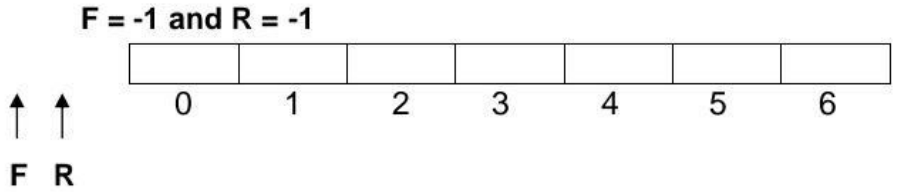


Fig. 2(a) Empty Queue

Queue

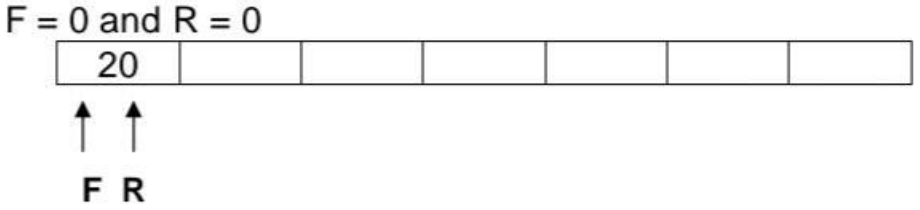


Fig. 2(b) One Element Queue

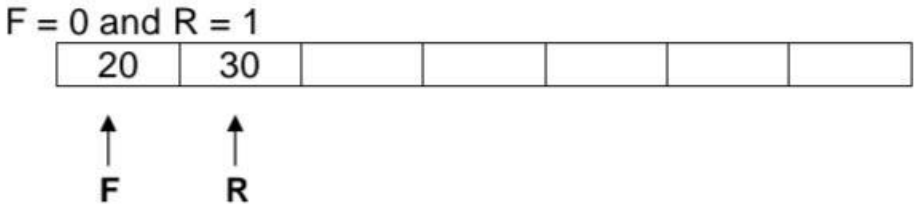


Fig. 2(c) Two Element Queue

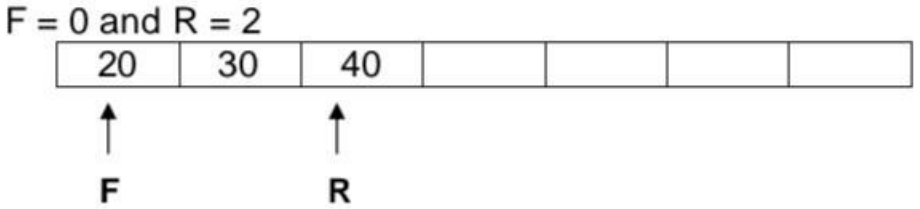


Fig. 2(d) Three Element Queue

Queue

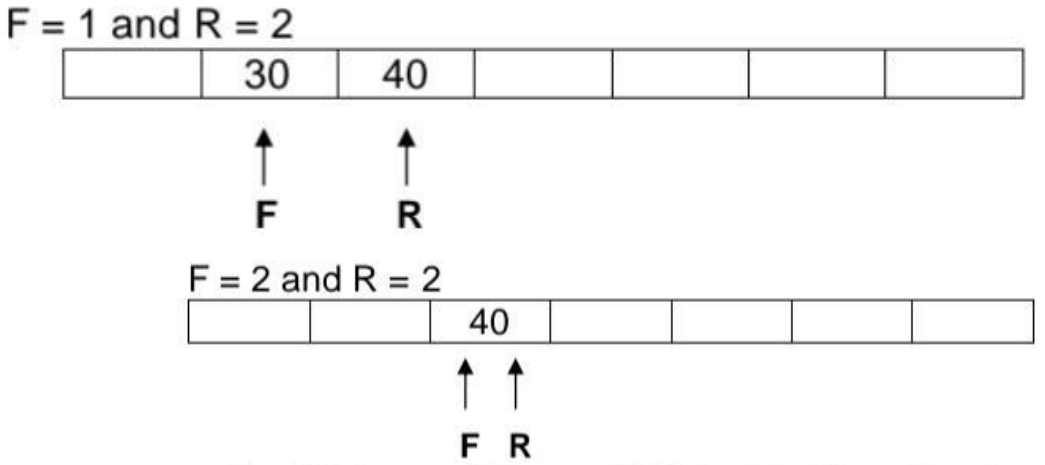


Fig. 2(f) Second Element (30) Deleted from Front

This is clear from Fig. 2(e) and 2(f), that whenever an element is removed from the queue, the value of Front is incremented by one i.e.,

Front = Front + 1

Now, if we insert any element in the queue, the queue will look like :

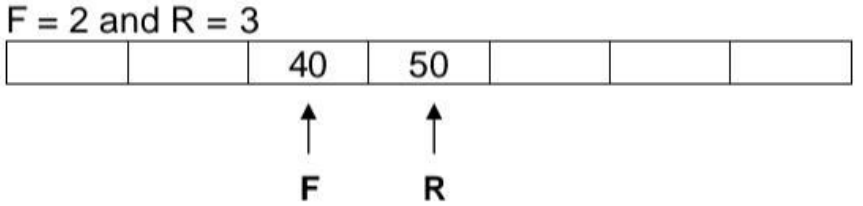


Fig. 2(g) Insertion after Deletion

Applications of Queue

- **Task Scheduling**: Queues can be used to schedule tasks based on priority or the order in which they were received.
- **Resource Allocation**: Queues can be used to manage and allocate resources, such as printers or CPU processing time.
- **Batch Processing**: Queues can be used to handle batch processing jobs, such as data analysis or image rendering.
- **Message Buffering**: Queues can be used to buffer messages in communication systems, such as message queues in messaging systems or buffers in computer networks.
- **Event Handling**: Queues can be used to handle events in event-driven systems, such as GUI applications or simulation systems.
- **Traffic Management**: Queues can be used to manage traffic flow in transportation systems, such as airport control systems or road networks.
- **Operating systems**: Operating systems often use queues to manage processes and resources. For example, a process scheduler might use a queue to manage the order in which processes are executed.
- **Network protocols**: Network protocols like TCP and UDP use queues to manage packets that are transmitted over the network. Queues can help to ensure that packets are delivered in the correct order and at the appropriate rate.
- **Printer queues** :In printing systems, queues are used to manage the order in which print jobs are processed. Jobs are added to the queue as they are submitted, and the printer processes them in the order they were received.
- **Web servers**: Web servers use queues to manage incoming requests from clients. Requests are added to the queue as they are received, and they are processed by the server in the order they were received.
- **Breadth-first search algorithm**: The breadth-first search algorithm uses a queue to explore nodes in a graph level-by-level. The algorithm starts at a given node, adds its neighbors to the queue, and then processes each neighbor in turn.

Queue Implementation using Array

- **Algorithm for Enqueue**

1)Start

2)Declare item

3) Check if rear =size-1, then go to step4 else go to step5

4) Print Queue overflow and go to step 10

5) Read the element to be inserted as item

6) Check if front=-1, then go to step7 else go to step8

7) Assign front as front+1

8) Set rear as rear+1

9) Assign queue[rear] as item

10) stop

Queue Implementation using Array

- Algorithm for Dequeue
- 1) start
- 2) Declare item
- 3) Check if front=-1 Or front>rear, then go to step4 else go to step5
- 4) Print Queue underflow go to step8
- 5) Set item=queue[front]
- 6) Assign front as front+1
- 7) print the deleted item as item
- 8) Stop

Queue Implementation using linked list

- Enqueue

1)start

2) Declare a structure pointer new

3) Allocate memory to new

4) Read the element to be inserted as item

5) Set new->data=item

6) set new->next =NULL

7) check if rear=NULL then go to step8 else go to step9

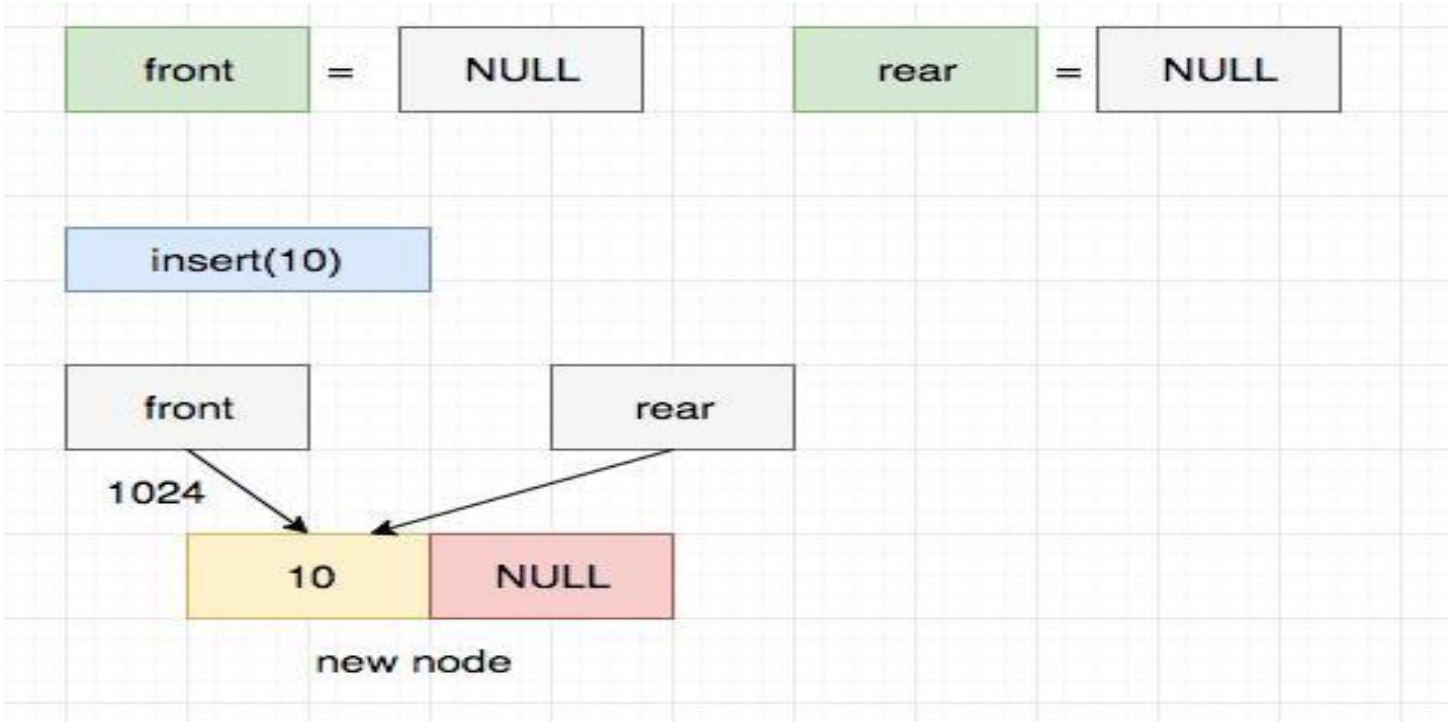
8)Set Front=rear=new

9) set rear->next=new

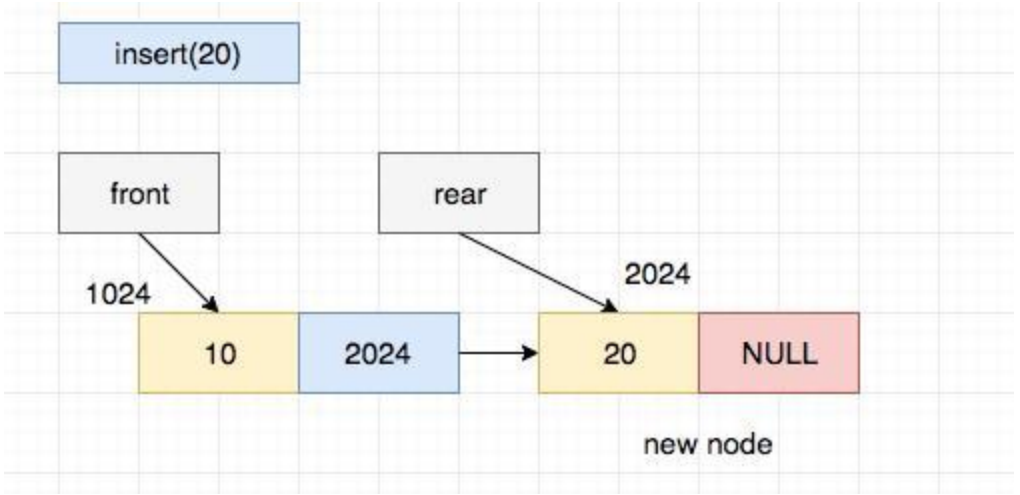
10) Set rear=new

11) stop

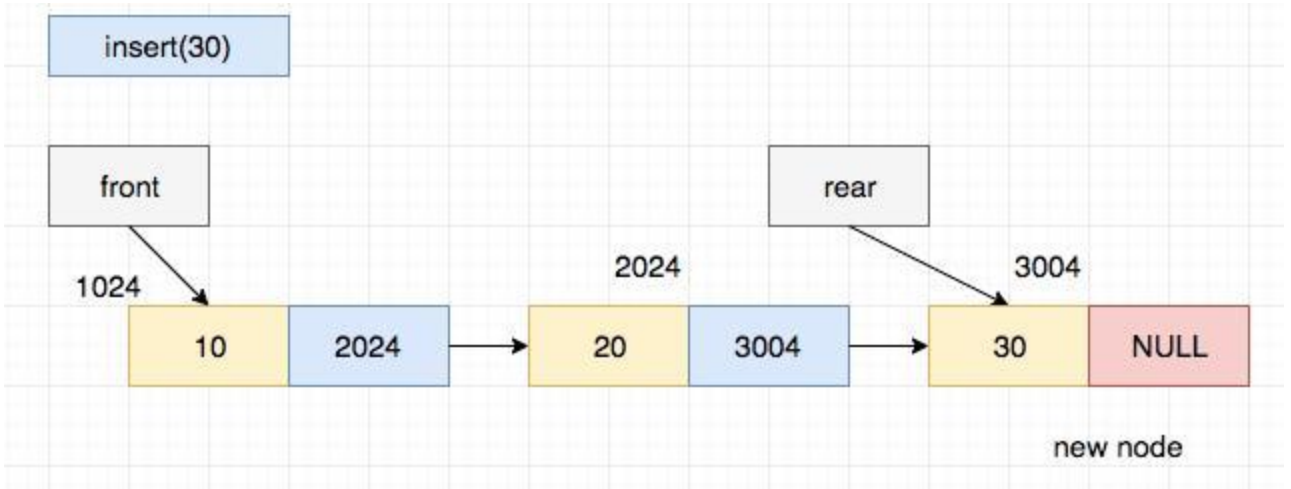
Queue Implementation using linked list



Queue Implementation using linked list



Queue Implementation using linked list

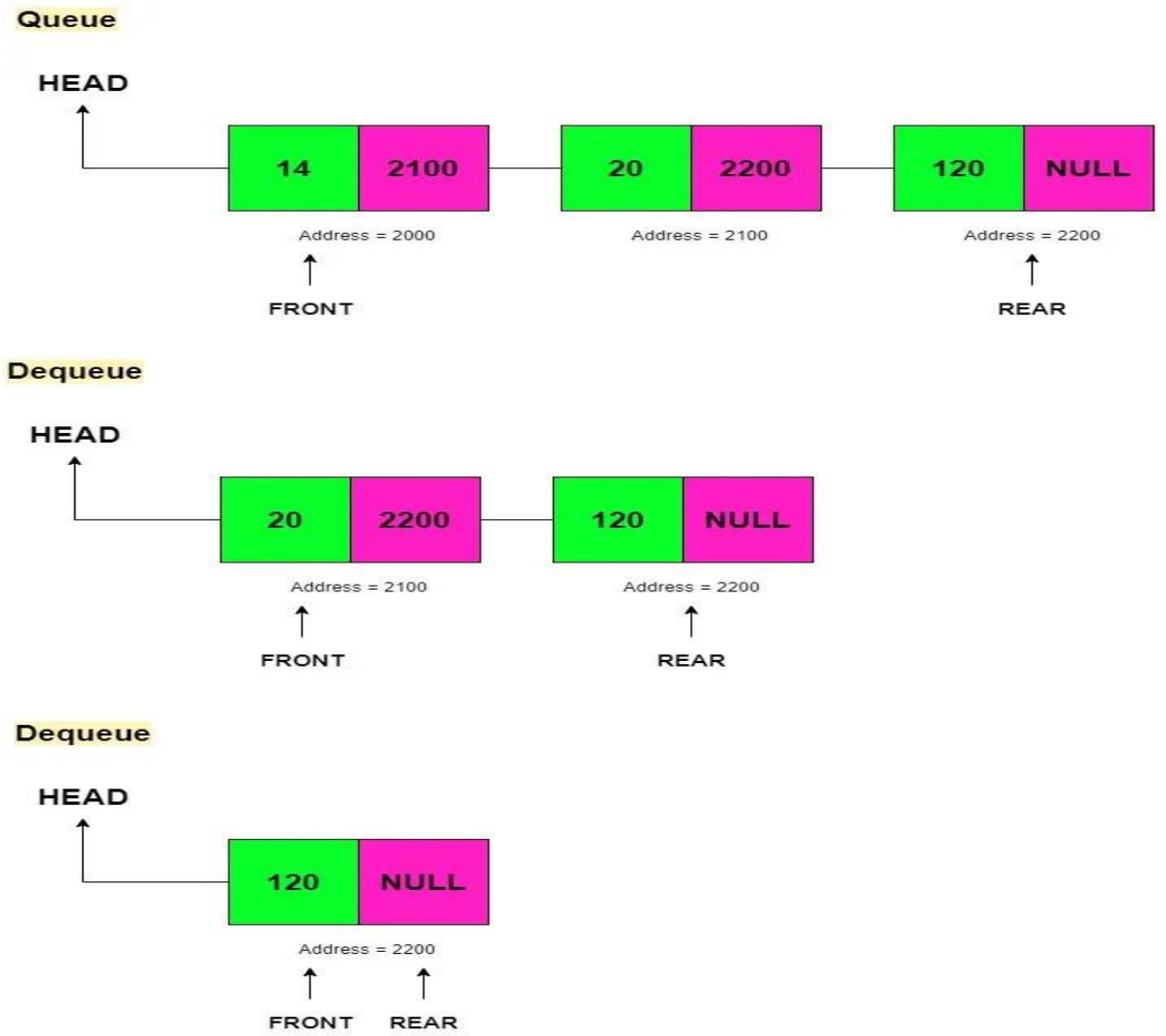


Queue Implementation using linked list

Dequeue

1) Start
2) Declare a structure pointer temp
3) Check if front=NULL, then print Queue underflow then go to step 10 else step4
4) Assign Temp as front
5) Check if front->next=NULL, then repeat step6-7 else go to step7
6) Set front=rear=NULL
7) Return temp->data
8) Set front=temp->next
9) Delete temp
10) stop

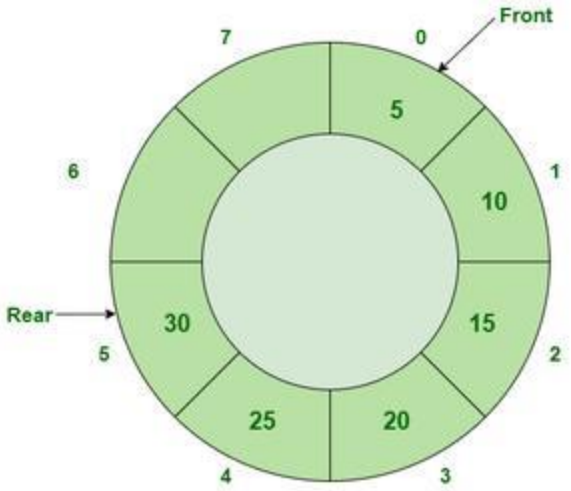
Queue Implementation using linked list



Circular Queue

Circular Queue is an extended version of a normal queue where the last element of the queue is connected to the first element of the queue forming a circle.

Circular queue
the rear end an
queue



eue items are added at
nt end of the circular

Circular Queue

In Circular Queues we utilize memory efficiently,because in queue when we delete any element only front increment by 1, but that position is not used later.

So when we perform more add and delete operation, memory wastage increase. But in Circular Queue memory is utilized, if we delete any element that position is used later, because it is circular.

Circular Queue

- In a linear queue with max. Size 5, after inserting element at the last location (4) of array, the elements can't be inserted, because in a queue the new elements are always inserted from the rear end, and rear here indicates to last location of the array (location with subscript 4) even if the starting locations before front are free. But in a circular queue, if there is element at the last location of queue, then we can insert a new element at the beginning of the array.
- REAR=(REAR + 1) % MAXSIZE.
- FRONT = (FRONT + 1) % MAXSIZE.

Priority Queue

Priority queue is a collection of elements where the elements are stored according to their priority levels. The order in which the elements get added or removed is decided by the priority of the element.

Following rules are applied to maintain a priority queue :

(1) The element with a higher priority is processed before any element of lower priority.

(2) If there are elements with the same priority, then the element added first in the queue

would get processed.

Priority queues are used for implementing job scheduling by the operating system where jobs with higher priorities are to be processed first.

Priority queues are often used in real-time systems

Properties of Priority Queue

- Every item has a priority associated with it.
- An element with high priority is dequeued before an element with low priority.
- If two elements have the same priority, they are served according to their order in the queue.